

Analog Interfacing

Chapter Contents

Overview	168
Introduction	168
Motivation	168
Concepts	168
Quantization	169
Sampling	171
Digital-to-Analog Conversion	172
Concepts	172
Converter Architectures	173
STM32F091RC DAC	173
Example Application: Analog Waveform Generator	174
Analog Comparator	176
Concepts	176
STM32F091RC Comparators	177
Input Configuration	178
Comparator Operation Configuration	179
Output Configuration	180
Interrupt Configuration	180
Example Application: Voltage Transition Monitor	181
Analog-to-Digital Conversion	184
Concepts	184
Converter Architectures	184
Inputs	185
Triggering	186
STM32F091RC ADC	186
Basic ADC Features	187
Advanced ADC Features	191
Example Applications	192
Hotplate Temperature Sensor	192
Infrared Proximity Sensor	195
Summary	202
Exercises	202
References	202

Overview

This chapter presents the concepts and methods that enable the interfacing of a digital microcontroller with analog circuitry. It covers quantization and sampling concepts, and then presents digital-to-analog conversion and the reverse. We examine examples such as waveform generation, temperature measurement, and proximity sensing using infrared energy.

Introduction

Motivation

Embedded computers often need to monitor the characteristics of the surrounding environment, such as sound, temperature, pressure, acceleration, strain, and light intensity. These characteristics are *analog* because they can take on an infinite number of possible values (even within a limited range). For example, a temperature sensor might indicate its reading by setting its output signal's voltage to 0.05 V/°C. A reading of 0.5 V would indicate a temperature of 10°C, whereas a reading of 0.50005 V would indicate 10.001°C. This analog signal must be converted to a digital (numerical) value for the program to process it; this is done with an analog-to-digital converter (ADC). Whether the ADC will be able to differentiate between these two temperatures depends on its resolution and other factors.

In order to generate sounds accurately (with little distortion), the MCU must generate analog voltage signals to drive headphones or speakers. The digital values representing the sound signal can be converted to an analog voltage using a digital-to-analog converter (DAC).

Concepts

Interfacing with analog devices involves **quantization** and **sampling**. To understand these concepts, let us consider how to generate a sound using an MCU and a speaker. We would like to drive the speaker with the signal shown in Figure 6.1. This signal varies continuously in both voltage and time, but an MCU cannot generate such a signal accurately for two reasons. First, there are *quantization* issues: the MCU is digital so it can generate only a limited (discrete) number of voltages on an output. Second, there are *sampling* issues: the MCU can update an output only at a limited rate, with some minimum time between updates. Our MCU's approximation of the desired output is limited by both quantization and sampling characteristics. These limits affect both digital signal processing [1] and digital control systems [2].

quantization

Process of selecting a discrete digital value to represent an analog value

sampling

Process of converting a continuous-time signal to a series of discrete-time samples

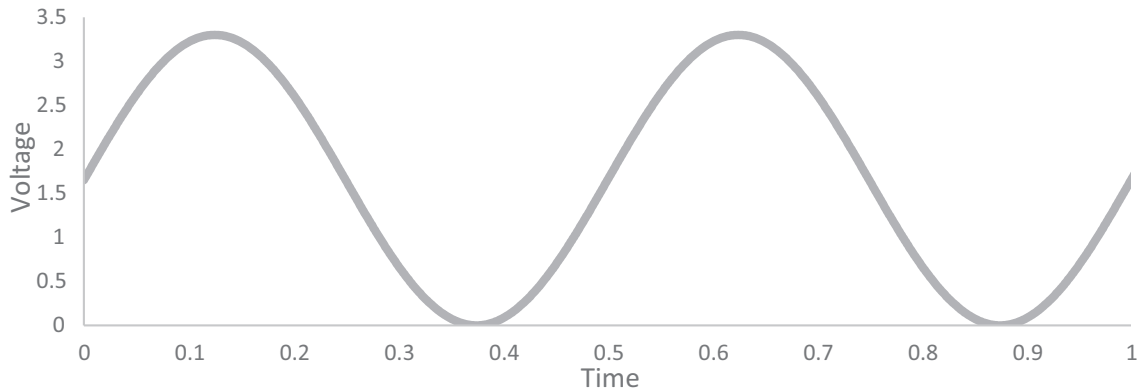


Figure 6.1 Sine wave signal has analog (continuous) voltage that varies continuously over time.

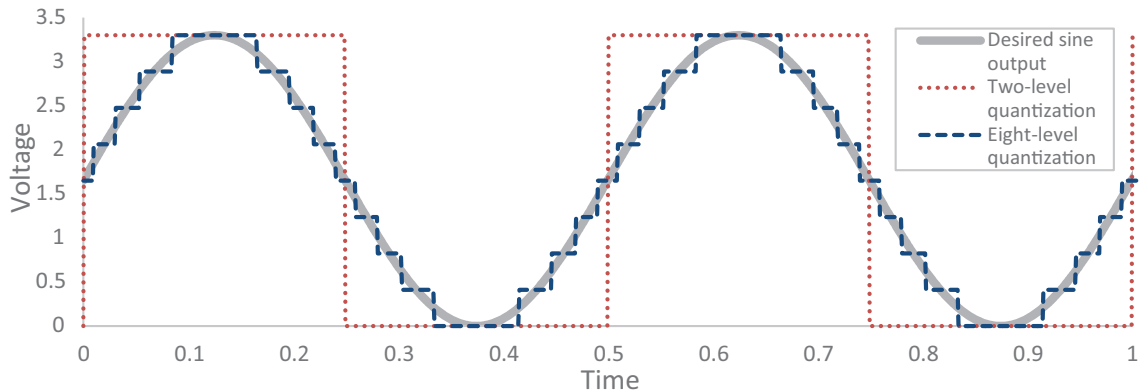


Figure 6.2 More quantization levels improves accuracy of generated sine wave.

Quantization

In Chapter 2 we learned about the digital general-purpose outputs of the MCU. If we use such an output in order to generate the sine wave, we will get the signal labelled “two-level quantization” shown in Figure 6.2. This is a rather inaccurate reconstruction of our desired sine wave.

The problem is that our output can generate only two possible voltage levels. If our MCU could generate more than two different voltage levels, then our output sine wave would be more accurate. The number of discrete values available for use defines the resolution of the quantization. In Figure 6.2, the signal labelled “eight-level quantization” shows the sine wave when generated with eight quantization levels.

An analog value can take on an infinite number of possible values along a continuous range. Quantization is the process of selecting one of multiple possible quantized (discrete) values to represent the analog value. Each quantized output value represents a range of possible analog input values. Figure 6.3 shows an example of quantization, identifying which output value represents each range of input values. For example, any voltage between 0.5 V and 0.75 V will be quantized to 2. The same code will be returned for 0.51 V and 0.74 V, making them indistinguishable to the MCU.

Input Voltage	Quantized Value		
	Decimal	Binary	
> 1V	3	11	Out of range
$V_{+ref} = 1\text{ V}$	3	11	
0.75 V	2	10	
0.5 V	1	01	
0.25 V	0	00	
$V_{-ref} = 0\text{ V}$	0	00	Out of range
< 0V	0	00	

Figure 6.3 Example of two-bit quantization of analog input voltage between 0 V and 1 V.

Two voltage references (V_{+ref} and V_{-ref}) are needed to define the boundaries of the conversion range. Often the positive supply rail (e.g. 3.3V) is used as the positive reference and ground is used as the negative reference.

Digital electronics work with binary values so the number of discrete output values is typically a power of 2. The resolution describes the number of bits (B) used to hold the output value. For example, a code with eight-bit resolution has $2^8 = 256$ possible output values. The example in Figure 6.3 shows a two-bit quantization.

Note that an output value n does not represent an exact voltage, but instead a range of voltages between V_{n_lower} and V_{n_upper} . For this example, 01 represents a voltage between 0.25 V and 0.50 V:

$$V_{n_lower} = \frac{n}{2^B} (V_{+ref} - V_{-ref}) = \frac{1}{4} (1 \text{ V} - 0 \text{ V}) = 0.25$$

$$V_{n_upper} = \frac{n+1}{2^B} (V_{+ref} - V_{-ref}) = \frac{2}{4} (1 \text{ V} - 0 \text{ V}) = 0.50$$

Note that the center of the voltage range for 01 is 0.375 V. The quantization approach might instead use an offset to center the voltage range on $n/2^B$. With this change, 01 would represent a voltage between 0.125 V and 0.375 V, with the center of the range at 0.25 V:

$$V_{n_lower} = \frac{n - \frac{1}{2}}{2^B} (V_{+ref} - V_{-ref}) = \frac{\frac{1}{2}}{4} (1 \text{ V} - 0 \text{ V}) = 0.125$$

$$V_{n_upper} = \frac{n + \frac{1}{2}}{2^B} (V_{+ref} - V_{-ref}) = \frac{1 + \frac{1}{2}}{4} (1 \text{ V} - 0 \text{ V}) = 0.375$$

As the resolution B increases, the difference between V_{n_lower} and V_{n_upper} decreases, so the quantization becomes more accurate. The maximum quantization error is typically half of this voltage range.

A *transfer function* defines the quantization mathematically and is typically provided in the documentation of the ADC or DAC. The following is an example of a simple transfer function with separate reference voltages:

$$n = \text{round}\left(\frac{V_{in} - V_{-ref}}{V_{+ref} - V_{-ref}} 2^B\right)$$

Sampling

We have just seen that better resolution for quantization improves our MCU's output signal accuracy. The other factor we need to consider is time: how often do we need to update an output or sample an input to get an adequate signal?

A sampled signal is a discrete-time representation (a series of individual samples) of a continuous-time signal. The sampling rate determines how often an input is measured, or how often an output is updated. Note that each sample may be an analog value (one of an infinite number of possible values) until it is quantized.

Any information between the samples is lost. Figure 6.4 shows that sampling the sine wave at a low frequency (slow sampling) results in a poor approximation. Raising the sampling rate (fast sampling) improves the approximation.

If the continuous-time signal changes more often than it is sampled, we will lose that high-frequency information. To understand this, let us consider the signal's frequency spectrum. Figure 6.5 shows the spectrum of a signal, with the horizontal axis representing frequency and the vertical axis showing power. The spectrum is symmetric across the 0 Hz frequency.

Sampling a signal in effect makes copies of the signal's spectrum centered at multiples of the sampling frequency f_s , as shown in Figure 6.6 and Figure 6.7. The Nyquist criterion states that if the signal has any energy at a frequency of $f_s/2$ or higher, those signal components will appear in the sampled signal at different (lower) frequencies, distorting the sampled signal with *aliasing*.

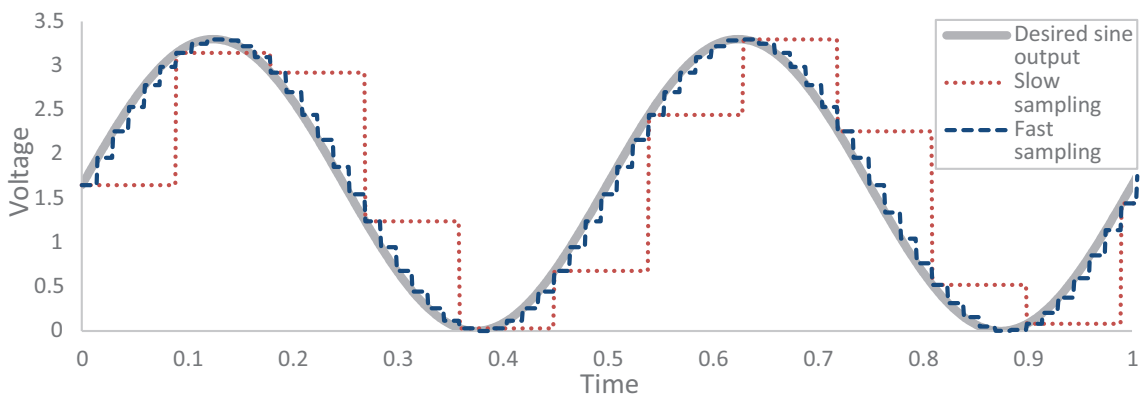


Figure 6.4 Faster sample rate improves accuracy of generated sine wave.

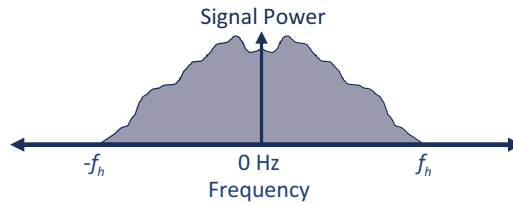


Figure 6.5 Signal spectrum shows distribution of power across frequencies.

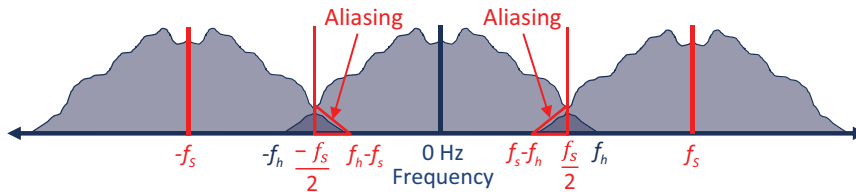


Figure 6.6 Sampling too slowly causes aliasing.

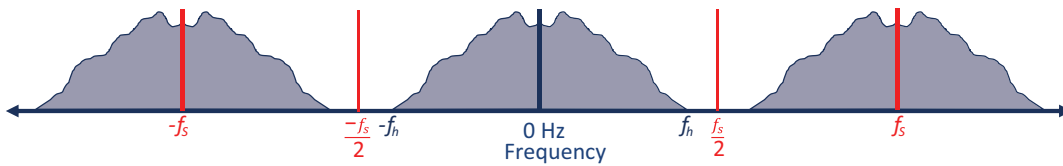


Figure 6.7 Raising sampling frequency f_s above signal's highest frequency component f_h eliminates aliasing.

Two complementary methods are used to prevent **aliasing**. First, the sampling frequency f_s is chosen to be more than twice the frequency of the highest signal frequency of interest f_h as shown in Figure 6.7. Second, a low-pass anti-aliasing filter is used to remove or greatly weaken the signal components above f_h . In order to simplify the design of the anti-aliasing filter, the sampling frequency f_s is often more than double f_h .

aliasing

Distortion of signal resulting from sampling at too low a frequency

Digital-to-Analog Conversion

Concepts

A digital-to-analog converter (DAC) generates an analog output signal based on the digital input value. The output signal may be a voltage or a current depending on the type of DAC. Here we will only consider voltage output DACs.

The minimum and maximum output voltages are defined by the DAC's lower and upper reference voltages. The lower reference voltage is often simply grounded at 0 V.

A transfer function defines the relationship between the digital input value n and the output voltage V_{out} . For a DAC with a lower reference voltage of 0 V, an upper reference voltage of $V_{+\text{ref}}$, and B bits of resolution, the general transfer function is:

$$V_{\text{out}} = n \frac{V_{+\text{ref}}}{2^B}$$

Converter Architectures

Two common DAC architectures are the resistor ladder and the R-2R ladder. An **N-bit resistor ladder** uses 2^N resistors of equal value connected in series between the upper and lower reference voltages. These resistors form a voltage divider with equally spaced voltages at the taps. An **R-2R resistor ladder** uses N resistors of one value (R) and N resistors of twice that value ($2R$).

Regardless of the type of DAC, an amplifier is typically used to buffer the output signal, enabling it to drive larger loads. This buffer amplifier is often located on-chip with the DAC to simplify application hardware design.

It is also possible to use a timer peripheral in pulse-width modulation (PWM) mode and a low-pass filter to create an analog output. We will discuss this in Chapter 7.

STM32F091RC DAC

The STM32F091RC has two DACs (shown in Figure 6.8), which can be configured with 8 or 12 bit resolution. There is one upper reference voltage (V_{DDA}) and one lower reference voltage (V_{SSA}) available. An amplifier buffer is available for the voltage output signal.

The control register DAC_CR is shown in Figure 6.9 and controls various aspects of the DAC.

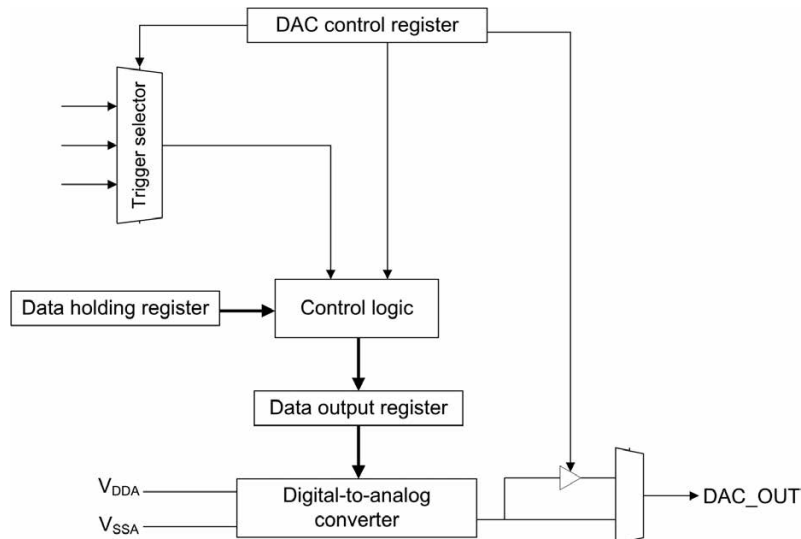


Figure 6.8 STM32F091RC DAC [3].

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	DMA UDRIE2	DMA EN2	MAMP2				WAVE2		TSEL2			TEN2	BOFF2	EN2
Reset			0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access			rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	DMA UDRIE1	DMA EN1	MAMP1				WAVE1		TSEL1			TEN1	BOFF1	EN1
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 6.9 DAC control register (DAC_CR).

- The DACx is enabled by writing a 1 to ENx in DAC_CR.
- The output buffer for the DACx is enabled by writing a 0 to BOFFx in DAC_CR.
- The DMAENx and DMAUDRIEx allows us to use the DAC using the Direct Memory Access (DMA) but we do not discuss them further here.

Output data for the DAC is either 12 bits or 8 bits long and is stored in the DAC_DORx (data output) register. This register is read-only and cannot be written into directly. Any data transfer needs to be done by loading one of the DAC_DHR (data hold) registers first. In 12-bit mode the data can be right- or left-aligned. The data is loaded in the DAC_DHR12Lx, DAC_DHR12Rx or DAC_DHR8Rx registers depending on the mode chosen: 12-bit left-align data (bits 15 to 4 of the register), 12-bit right-align data (bits 11 to 0 of the register) or 8-bit right-align data (bits 7 to 0 of the register) respectively. It is also possible to use both DACs at the same time in dual mode. In this case the data is written in the DAC_DHR12LD, DAC_DHR12RD or DAC_DHR8RD registers.

The transfer function is similar to the general DAC transfer function:

$$V_{\text{out}} = n \frac{V_{\text{DDA}}}{4096}$$

The data from the DAC_DHR register is transferred automatically to the DAC_DORx register. If a trigger is enabled by setting the TENx field of DAC_CR and selected with the TSELx field of DAC_CR (it can be a hardware or software trigger), the transfer happens only after the trigger occurs.

Example Application: Analog Waveform Generator

Let's use the DAC to create a simple analog waveform generator. The output for DAC1 is connected to pin PA4, as shown in Figure 6.10.

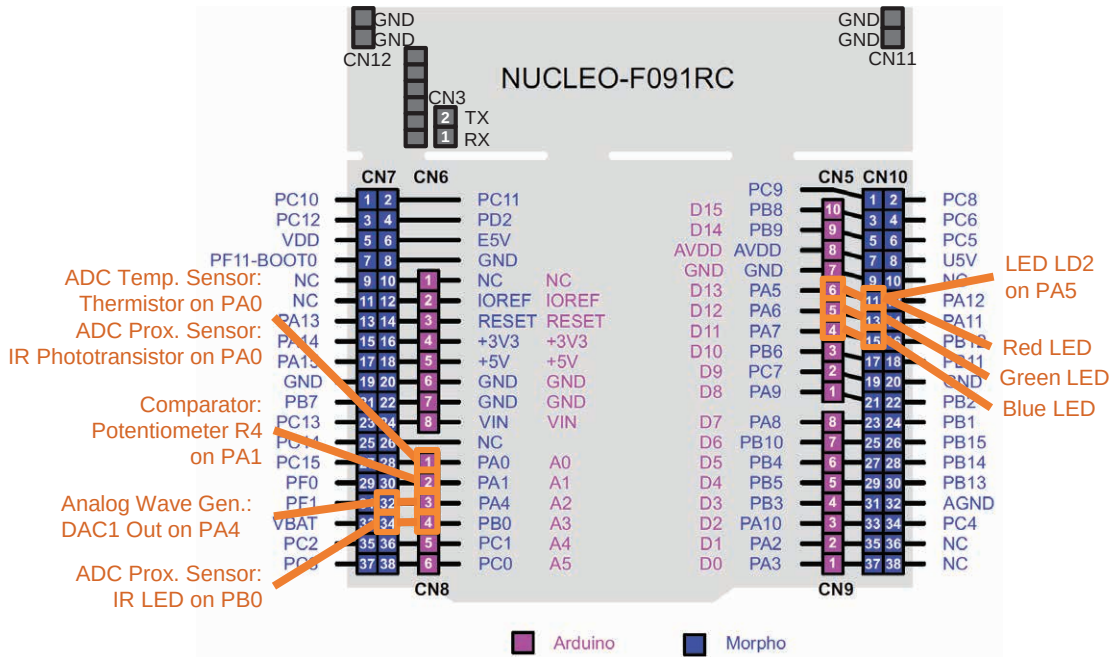


Figure 6.10 Locations of signals used for example code in this chapter.

Listing 6.1 shows the function `Init_DAC` which initializes the DAC and related peripherals.

```
void Init_DAC(void) {
    // Enable clocks for DAC and PA4
    RCC->APB1ENR |= RCC_APB1ENR_DACEN;
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    // Init PA4 as analog by setting both MODER bits
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER4, ESF_GPIO_MODER_ANALOG);

    // Init DAC1, connecting it to PA4
    DAC1->CR = DAC_CR_EN1;
}
```

Listing 6.1 Function to initialize DAC.

The second function `Triangle_Output` (in Listing 6.2) sweeps the DAC output voltage up and down repeatedly. Note that STM32F091RC DACs have a mode to automatically generate a triangle wave that could be used to generate the same wave.

```
#define MAX_DAC_CODE 4095

void Triangle_Output(void) {
    int i = 0;
    int change = 1;
    while (1) {
        DAC->DHR12R1 = i;
        i += change;
```

```

    if (i <= 0)
        change = 1;
    else if (i >= MAX_DAC_CODE - 1)
        change = -1;
    }
}

```

Listing 6.2 Function to generate triangle wave output.

When the function `Triangle_Output` is called, the system creates the waveform shown in Figure 6.11.

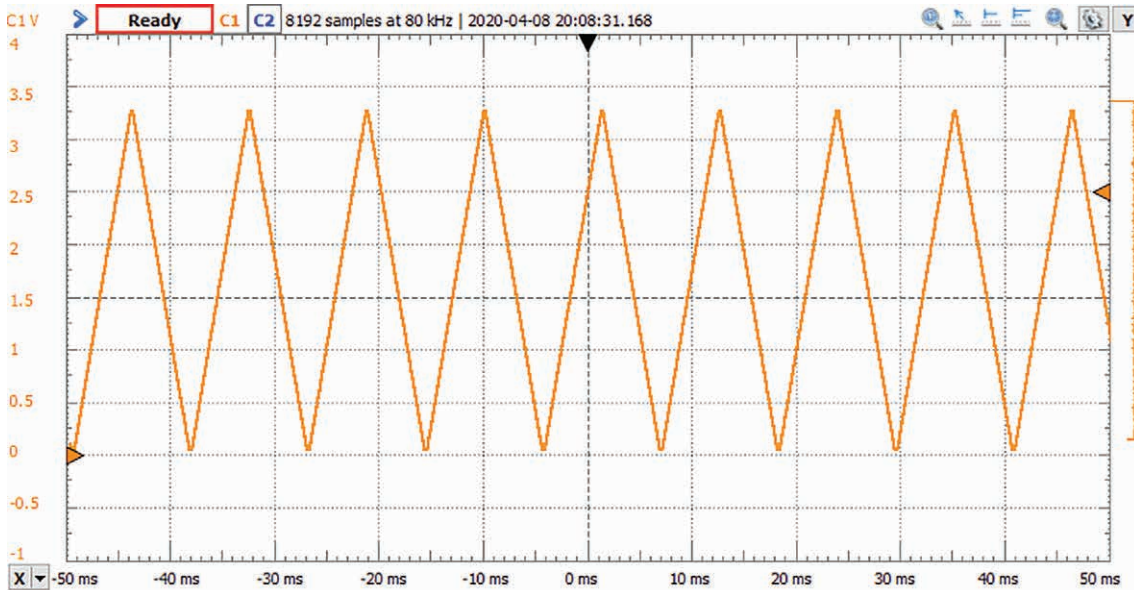


Figure 6.11 Triangle analog voltage waveform created by DAC and function `Triangle_Output`.

Analog Comparator

Concepts

Figure 6.12 shows an analog **comparator**, which is a circuit that compares two analog voltages and indicates which is greater. This can be used to determine if a voltage is above or below a given level. The comparator has two inputs that are labelled plus and minus. We apply a voltage to each input: V_{inP} to plus, V_{inM} to minus. If $V_{inP} > V_{inM}$, then the comparator output will be a logic one. Otherwise the output will be a logic zero. The program can read the comparator output directly with software. Most comparators can generate an interrupt request when their output changes.

comparator

Circuit which compares two analog inputs to identify larger value

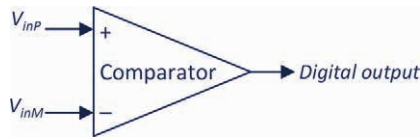


Figure 6.12 Analog comparator output is 1 if input voltage $V_{inP} > V_{inM}$, 0 otherwise.

Connecting one of the inputs to a known reference voltage enables us to determine if the other input is above or below that voltage. Using the comparator this way quantizes an input voltage into one of two possible values, zero or one, providing one bit of data. Some MCUs supplement their comparators with multiple fixed reference voltages, or even a DAC for greater flexibility.

STM32F091RC Comparators

The comparator peripherals of the STM32F091RC MCU are shown in Figure 6.13. In this section we examine the key features of the peripherals; other features such as window mode are described in the reference manual [4].

In order to enable the comparators, the clock gating must be enabled by writing a one to the SYSCFGCOMPEN bit in the RCC_APB2ENR register, and then writing one to the comparator enable bit (COMPxEN) in COMP_CSR, shown in Figure 6.14.

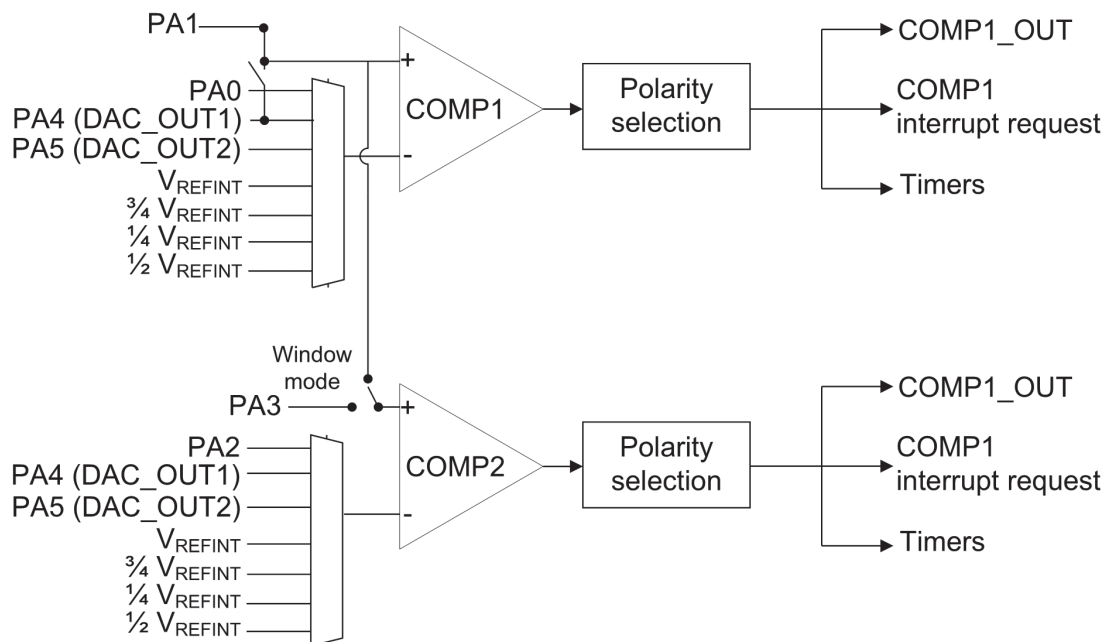


Figure 6.13 Overview of comparator peripherals in STM32F091RC MCU [3]. The two comparators (COMP1 and COMP2) are supplemented with input multiplexers and output processing logic. A DAC can be connected to the input to generate a reference voltage.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	COMP2 LOCK	COMP2 OUT	COMP2HYST		COMP2 POL	COMP2OUTSEL			WNDW EN	COMP2INSEL			COMP2MODE		Res.	COMP2 EN
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0
Access	rw*	r	rw*	rw*	rw*	rw*	rw*	rw*	rw*	rw*	rw*	rw*	rw*	rw*		rw*

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	COMP1 LOCK	COMP1 OUT	COMP1HYST		COMP1 POL	COMP1OUTSEL			Res.	COMP1INSEL			COMP1MODE		COMP2 SW1	COMP1 EN
Reset	0	0	0	0	0	0	0	0		0	0	0	0	0	0	0
Access	rw*	r	rw*	rw*	rw*	rw*	rw*	rw*		rw*	rw*	rw*	rw*	rw*	rw*	rw*

Figure 6.14 COMP_CSR controls general comparator settings and status.

Input Configuration

Each comparator has two inputs (INP and INM), as shown in Figure 6.13.

The plus input INP can be connected to one input bit: PA1 for COMP1, and usually PA3 for COMP2. To do this, we need to configure the GPIOAy pin for the analog mode by setting GPIOA_MODER_MODERy to 3, as shown in Figure 6.15. Further information is in Chapter 2.

The minus input INM sets the value against which the plus input is compared, so it offers more options. The COMPxINSEL field of COMP_CSR (shown in Figure 6.14) determines which input is selected according to Table 6.1.

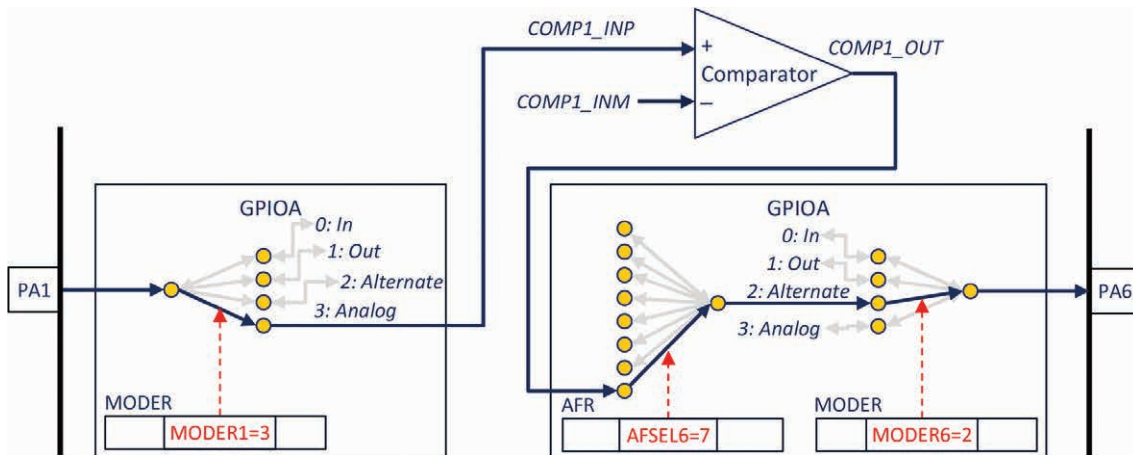


Figure 6.15 Configuring GPIO to connect comparator input and output to pins for PA1 and PA6. Note that for many applications, comparator output on a port pin is not needed.

- There are four fixed voltage references available. The MCU contains an embedded bandgap voltage reference called V_{REFINT} with an output of approximately 1.23 V [3,5]. This voltage is divided down to provide three additional reference levels ($V_{\text{REFINT}}/4$, $V_{\text{REFINT}}/2$, and $V_{\text{REFINT}}*3/4$).
- Three inputs are connected to Port A pins, allowing external circuits to provide analog reference voltages. Two of these pins (PA4 and PA5) can be configured as DAC outputs, allowing the system to generate custom reference voltages with 12-bit resolution without external circuitry.

Table 6.1 Comparator Input Multiplexer Settings

INSEL	Input selected for COMP1/COMP2	MCU signal for COMP1/COMP2
000	1/4 of VREFINT	–
001	1/2 of VREFINT	–
010	3/4 of VREFINT	–
011	VREFINT	–
100	COMP1_INM4/COMP2_INM4 (DAC_OUT1)	PA4
101	COMP1_INM5/COMP2_INM5 (DAC_OUT2)	PA5
110	COMP1_INM6/COMP2_INM6	PA0/PA2
111	Reserved	–

Comparator Operation Configuration

The operation of the comparator can be configured in various ways with the COMP_CSR register (in Figure 6.14).

COMPxHYST selects one of four possible hysteresis settings (from none to 31 mV). Hysteresis makes circuits less vulnerable to noisy analog input signals. Consider driving COMP1_INM with 1.23 V and COMP1_INP with a clean signal which ramps up from 1.20 to 1.30 V. The output will be 0 until COMP1_INP crosses 1.23 V, when the output will switch to 1. Now consider adding a fast sine wave with 0.01 V of noise to that clean ramp. Now COMP1_INP may cross 1.23 V several times, causing the output signal to switch for each crossing.

Configuring the comparator to use 0.02 V of hysteresis changes the threshold voltages:

- COMP1_INP must rise above 1.23 V for the output to switch from 0 to 1
- COMP1_INP must fall below $1.23 \text{ V} - 0.02 \text{ V} = 1.21 \text{ V}$ for the output to switch from 1 to 0

The 0.01 V of noise is less than the 0.02 V of hysteresis, so the output will switch once.

COMPxMODE selects one of four possible switching speeds for the comparator. Faster switching increases power consumption, so this field allows the developer to balance speed and power use.

There are further features available which are described in the reference manual [4]. The two comparators can be linked together (in window mode) to indicate if V_{inP} is between the two references. The control registers can be locked, preventing corruption by other software (at least until the processor is reset).

Table 6.2 Possible Comparator Digital Output Signal Locations

MCU signal	Signal	Alternate function
PA0	COMP1_OUT	AF7
PA1	COMP2_OUT	AF7
PA6	COMP1_OUT	AF7
PA7	COMP2_OUT	AF7
PA11	COMP1_OUT	AF7
PA12	COMP2_OUT	AF7

Output Configuration

The comparator output is normally 1 if $COMPx_INP > COMPx_INM$. However, the output's polarity can be inverted (0 under these conditions) by writing a 1 to the $COMPxPOL$ field of the $COMP_CSR$ register shown in Figure 6.14.

Polling software can determine the comparator's output by reading the $COMPxOUT$ field of the $COMP_CR$ register shown in Figure 6.14.

Each comparator's output signal can drive a digital output pin (listed in Table 6.2). To configure the output connection (as shown in Figure 6.15), we need to configure the $GPIOAy$ pin for its alternate function mode by setting the $MODERy$ field of $GPIOA_MODER$'s to 2. Then we select the specific alternate function by writing its alternate function code (7) to the $AFSELy$ field of $GPIOA_AFR$. This is shown in Listing 6.4. Again, further information is in Chapter 2.

Interrupt Configuration

Each comparator is connected to the extended interrupts and events controller (EXTI), shown in Figure 6.16. The $COMP1$ output is connected to the EXTI line 21 and the $COMP2$ output is connected to the EXTI line 22. To enable the interrupt on one of these lines, set the corresponding bit (21 or 22) of the Interrupt Mask Register ($EXTI_IMR$). The interrupt can be triggered on the rising edge by setting the corresponding bit of the Rising Trigger Selection Register ($EXTI_RTSR$) and the interrupt can be triggered on the falling edge by setting the corresponding bit of the Falling Trigger Selection Register ($EXTI_FTSR$).

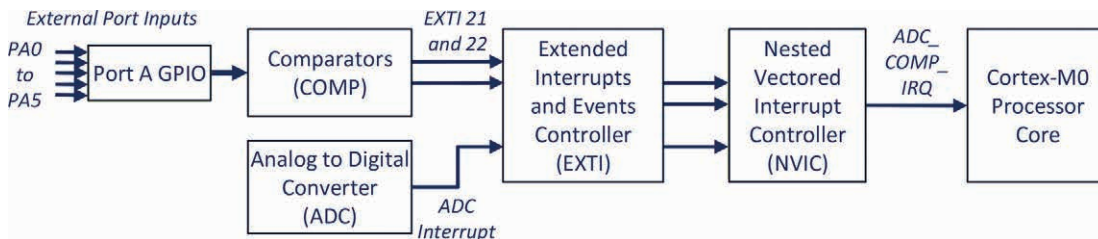


Figure 6.16 To use comparator interrupts, configure EXTI and NVIC. NVIC groups comparator interrupts with ADC interrupt to use a single handler ADC_COMP_IRQ .

Hardware sets the bit in the Pending Register (EXTI_PR) to a one while an interrupt is pending. The pending bit needs to be reset by writing a 1 in the EXTI_PR during the ISR. Until the pending bit is reset, the ISR will be triggered repeatedly.

The NVIC is designed so that the interrupts of COMP1, COMP2, and the Analog-to-Digital Converter (discussed shortly) are handled by a single handler: `ADC1_COMP_IRQHandler`. As a result, the handler needs to determine the source of the interrupt, service it, and then clear its pending bit.

Example Application: Voltage Transition Monitor

Let's see how to use the comparator to monitor an analog input voltage. We can create an adjustable voltage divider by connecting a potentiometer between 3.3 V and 0 V, shown in Figure 6.17 and Figure 6.18. We feed the signal from the potentiometer's middle (wiper) contact to PA1 which will connect to comparator input COMP1_INP. COMP1_INM sets the threshold voltage and will be connected to $\frac{1}{2}$ VREFINT.

We will use the comparator in two ways. First, the **hardware** signal from the comparator will be used to light the green LED when the COMP1_INP is above the threshold. Second, the comparator will generate an **interrupt** whenever COMP1_INP crosses the threshold. Crossing the threshold in a rising direction will light the red LED and extinguish the blue LED, while falling does the opposite. After reset, the LED will either be off or green based on COMP1_INP. No interrupts have occurred yet, so the red and blue LEDs are off. After the first threshold crossing, the interrupt handler runs and either the red or blue LED is lit.

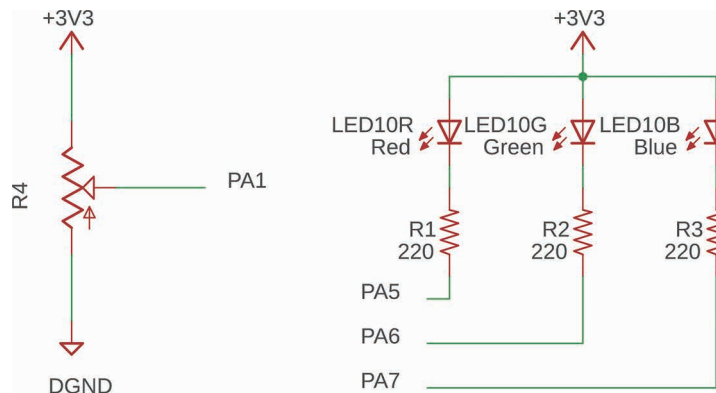


Figure 6.17 Voltage transition monitor circuit uses potentiometer R4 to generate voltage for comparator 1 positive input, uses RGB LEDs for output.

The `main` function is shown in Listing 6.3. Note that we use special versions of the RGB functions which let us control the red and blue LEDs with GPIO, letting the comparator drive the green LED.

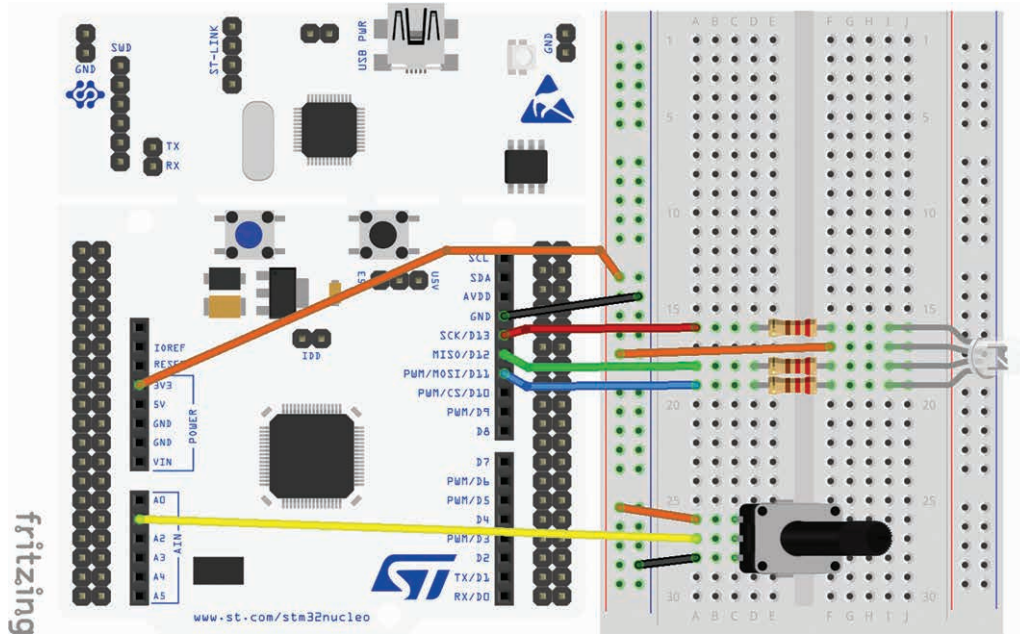


Figure 6.18 Parts placement for voltage transition monitor.

```
int main(void) {
    // Initialize red and blue LEDs for GPIO control, initially off
    Init_GPIO_Discrete_RGB(1, 0, 1, 0);
    // Initialize comparator, which will drive green LED
    Init_Comparator();
    while (1)
    ;
}
```

Listing 6.3 main function for comparator demonstration.

We initialize the comparator 1 as shown in Listing 6.4. After enabling peripheral clocks, the code connects the comparator to its input and output signals. We connect PA1 to COMP1_INP by setting GPIOA_MODER's MODER1 field to ESF_GPIO_MODER_ANALOG (3), which sets PA1's circuitry to analog mode.

We connect COMP1_OUT to PA6 to drive the green LED by setting PA6 to use an alternate function (by writing ESF_GPIO_MODER_ALT_FUNC (2) to the MODER6 field of GPIOA_MODER) and select the COMP1_OUT alternate function (by writing 7 to the AFSEL6 field of GPIOA_AFR).

This hardware signal will operate correctly even with most software failures or bugs, making it useful for development (when much of the system is likely to be buggy) and also safety-critical systems (for example, to implement over-voltage protection).

```
void Init_Comparator(void) {
    // Enable peripheral clock for COMP (and SYSCFG)
    RCC->APB2ENR |= RCC_APB2ENR_SYSCFGCOMPEN;
    // Enable peripheral clock of GPIOA
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
```



```

// Configure PA1 as analog input to comparator
// Set bit 1 MODER field to b11 for analog
// (ESF_GPIO_MODER_ANALOG = 3)
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER1, ESF_GPIO_MODER_ANALOG);

// Configure PA6 as digital output from comparator
// Set bit 6 MODER field to b10 for alternate function
// (ESF_GPIO_MODER_ALT_FUNC = 2)
MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER6, ESF_GPIO_MODER_ALT_FUNC);

// Select CMP1_OUT as alternate function for PA6.
// PA6 is in lower byte of port, so use AFRL, access as AFR[0]
MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFRL_AFRL6, 7);

// Comparator initialization
MODIFY_FIELD(COMP->CSR, COMP_CSR_COMP1EN, 1);
MODIFY_FIELD(COMP->CSR, COMP_CSR_COMP1INSEL, 1);

// Interrupt initialization
// Comparator 1: EXTI line 21
EXTI->FTSR |= EXTI_FTSR_TR21; // Enable falling trigger
EXTI->RTSR |= EXTI_RTSR_TR21; // Enable rising trigger
EXTI->IMR |= EXTI_IMR_MR21;   // Set mask bit to enable int.

// Enable IRQ in NVIC, shared by ADC and COMPS
NVIC_SetPriority(ADC1_COMP_IRQn, 128);
NVIC_ClearPendingIRQ(ADC1_COMP_IRQn);
NVIC_EnableIRQ(ADC1_COMP_IRQn);
}

```

Listing 6.4 Code to configure comparator to detect input voltage crossing 1/2 VREFINT level.

```

void ADC1_COMP_IRQHandler(void) {
    if (EXTI->PR & EXTI_PR_PR21) { // COMP_EXTI_LINE_COMP1
        EXTI->PR = EXTI_PR_PR21; // Clear pending request flag
        if ((COMP->CSR & COMP_CSR_COMP1OUT) == 0)
            // Level of PA1 is lower than 1/2 Vref, so light red LED
            Control_Discrete_RGB_LEDs(1, -1, 0);
        else
            // Level of PA1 is higher than 1/2 Vref, so light blue LED
            Control_Discrete_RGB_LEDs(0, -1, 1);
    }
}

```

Listing 6.5 Interrupt handler to light red LED below ½ VREFINT, blue LED above it.

The interrupt handler shown in Listing 6.5 confirms that COMP1 triggered the interrupt. It then clears the pending flag to prepare for the next transition. The handler then checks the COMP1OUT field to determine if the input voltage is lower or higher than the reference voltage, and lights the LEDs accordingly using `Control_Discrete_RGB_LEDs`. This function (located in `rgb.c` in the code repository) accepts an integer argument for each LED. A zero turns off the LED, a positive value turns on the LED and a negative value leaves the LED unchanged. Note the argument for the green LED is always negative, so the code does not try to control the LED directly.

Analog-to-Digital Conversion

Concepts

Like an analog comparator, an Analog-to-Digital Converter (ADC) quantizes an analog input voltage to create a binary output code. The main difference is that it provides more than two quantization levels and therefore more bits of resolution, allowing higher-quality measurements of analog values.

Converter Architectures

There are various approaches to building an ADC. We will discuss the flash and successive approximation architectures. There are others as well (e.g. sigma-delta, dual-slope integrating) but we will not discuss them here.

The comparator we saw earlier is essentially a 1-bit ADC. A B -bit ADC can be built out of 2^B analog comparators operating in parallel, each with a different reference voltage. The resulting B -bit code is created with digital logic that encodes the output bits of the 2^B comparators. This is called a flash architecture because it is extremely fast. The conversion time consists of the comparator delay and the digital encoder delay. However, this approach requires many comparators: increasing the resolution by one bit doubles the number of comparators needed. This increases power use and circuit area and therefore cost.

We can use a single comparator to make a series of comparisons, changing its reference voltage for each comparison. The successive approximation architecture uses this approach and performs a binary search to quantize the input. Figure 6.19 shows the hardware for this converter, including a successive approximation register (SAR), a DAC, an analog comparator and control logic.

Figure 6.20 shows how the converter of Figure 6.19 works. The converter first clears all bits in the SAR to zero. It then sets the most significant bit in the SAR to one. The comparator determines if the input voltage is greater than the DAC output voltage. If so, the first bit is left as one, or else it is cleared to zero. This process advances to the next bit and repeats until all bits have been determined.

A successive approximation ADC is not as fast as a flash ADC, as it requires one comparison for each bit of the result. However, the circuitry is much smaller and does not grow quickly as resolution is increased. Adding one bit of resolution slows down the conversion slightly, as it requires one more comparison. However, the circuit area increase is marginal. Because of these positive characteristics, most MCUs with built-in ADCs use a successive approximation ADC.

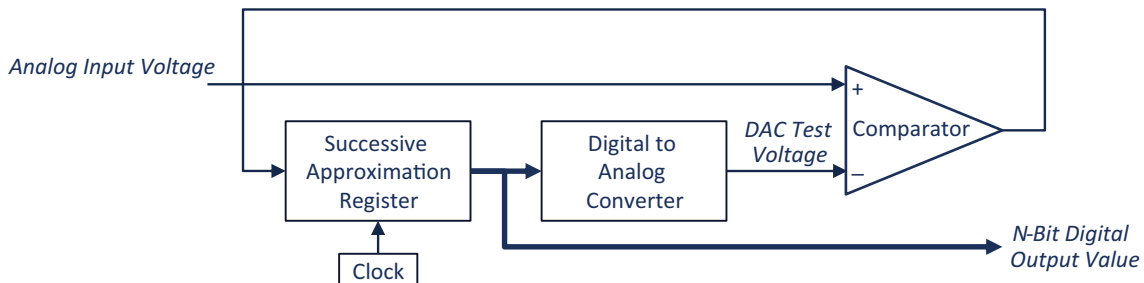


Figure 6.19 Architecture of successive approximation ADC.

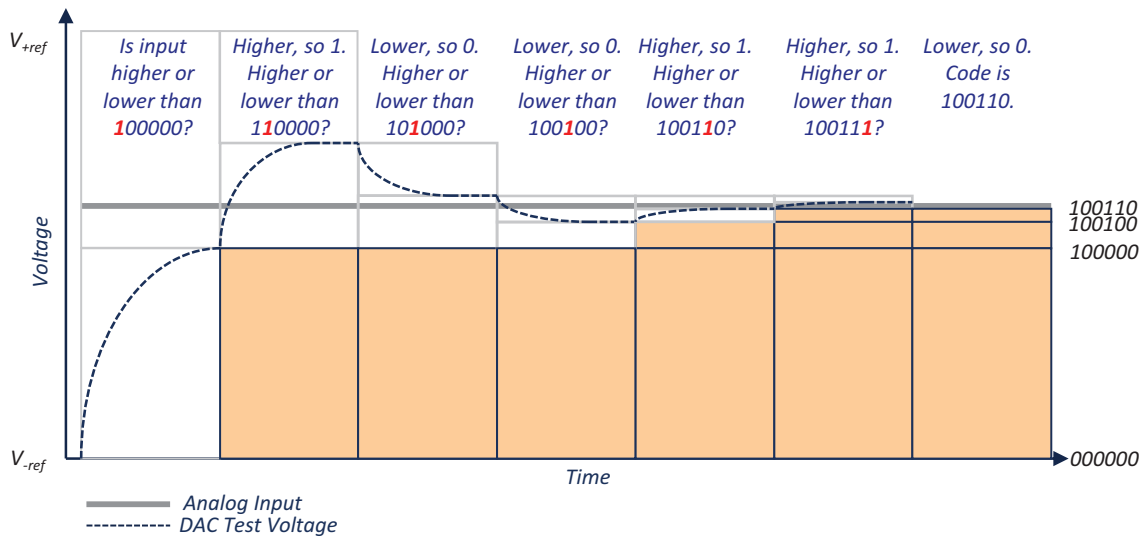


Figure 6.20 Performing analog-to-digital conversion with successive approximation.

Inputs

ADCs often include an input multiplexer to allow a single ADC to select one of the multiple input channels to measure. We store a channel select code in a control register to specify the input channel.

An ADC requires two voltage references to define the conversion range. These voltage references are used in the transfer function. Often the positive supply rail (e.g. 3.3V) is used as the positive reference, and ground is used as the negative reference.

Many types of ADC (including those using successive approximation) will produce incorrect results if the input changes much during the conversion process. A sample and hold circuit can be used to sample the input signal and then hold it fixed during the conversion time, eliminating this source of error. Conceptually, this circuit consists of a capacitor and a switch. Figure 6.21 shows the operation of the circuit. When the switch is closed, the circuit will **sample** the input by charging the capacitor to the input voltage. Opening the switch disconnects the capacitor from the input, so the circuit will **hold** the saved value of the input voltage for the ADC to perform its conversion. The capacitor does not charge instantaneously when in sample mode but is limited by the resistance of the input voltage source and switch and the capacitance. As a result, the switch must be closed for a minimum sample time.

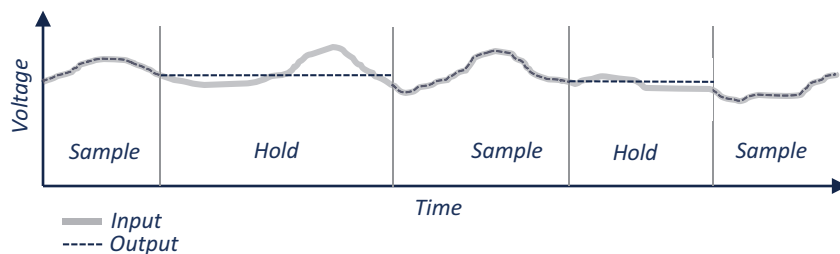


Figure 6.21 Sample and hold circuit tracks input voltage or holds last value depending on mode.

A single-ended signal represents information with the voltage difference between the signal and ground. Differential signals represent information with the voltage difference between two signals, neither of which is ground. This reduces the effects of noise. Some ADCs support differential signal inputs. These ADCs contain hardware that allows direct measurement of the voltage difference, making conversion a single-step process. An ADC without differential input signal support needs to convert each of the two signals separately, and then use software to find the difference.

Triggering

The trigger is a signal that tells the ADC to sample an input and then hold and convert it. An ADC will typically include two types of triggers: software and hardware. A software trigger requires the software (or DMA, discussed in Chapter 9) to write a value to a specific ADC control register to start the conversion. A hardware trigger requires a hardware signal to be asserted by a circuit, whether outside the MCU or within it. For example, a hardware timer could generate a signal every millisecond to trigger the ADC operation.

The ADC performs sampling and conversion and then indicates that the conversion has completed. This is done by setting a flag in an ADC status register, and possibly also signalling an interrupt request. At this point the result of the conversion is available in digital form in an ADC result register.

STM32F091RC ADC

The STM32F091RC MCU contains an ADC with many features; an overview appears in Figure 6.22. In this section we will examine the basic features. Full details can be found in the ADC chapter of the MCU reference manual [3] and in the MCU data sheet [5]. Note that Appendix A of the MCU reference manual provides many sample code listings for configuring and using the ADC [3].

As with other peripherals, the ADC's peripheral clock must be enabled (by setting ADCEN in RCC_APB2ENR) before configuring the ADC.

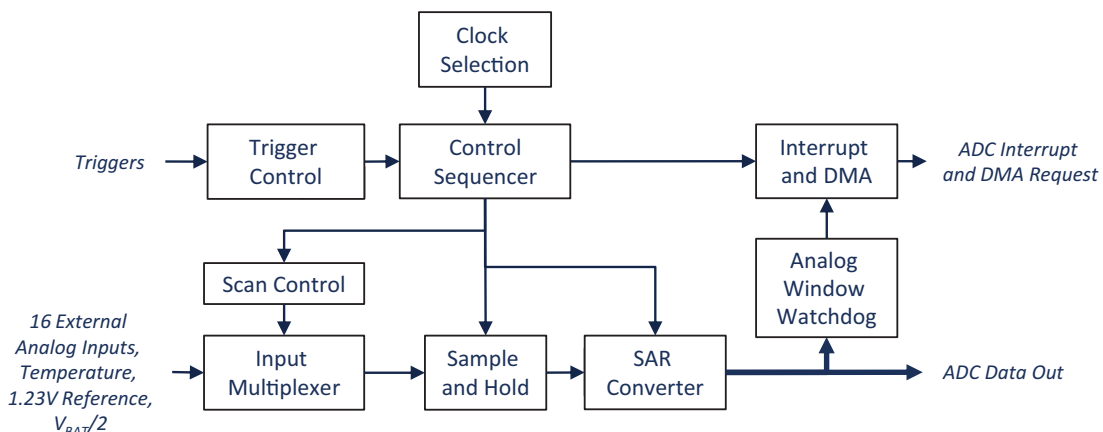


Figure 6.22 Overview of STM32F091RC ADC.

The basic ADC operation involves selecting an input channel with the multiplexer, awaiting a trigger event to start the conversion, awaiting the conversion completion, and then processing the result.

Basic ADC Features

ADC Control

The ADC Control Register (ADC_CR) has five fields which control basic ADC behavior.

- Writing 1 to the ADEN field will enable the ADC. It takes time for the ADC to stabilize, so the code should test the ADRDY flag in AD_ISR repeatedly until it becomes 1 (set by hardware) when the ADC finally is ready for use. Listing 6.6 shows an example.
- Writing 1 to the ADDIS field disables and powers down the ADC.
- Writing 1 to the ADSTART field allows the ADC to perform conversions in response to triggers.
- Writing 1 to the ADSTP field stops the ADC from performing conversions.
- Writing 1 to the ADCAL field starts calibration of the ADC, which is useful for high-precision applications. Please see the reference manual for further details.

Voltage References

The ADC uses two voltages references to define its input range. The analog supply voltage V_{DDA} (nominally 3.3 V) is used as the upper reference voltage. The analog ground V_{SSA} (0 V) is used as the lower reference voltage.

Conversion Clock

The conversion clock signal ADC_CLK determines how quickly the ADC samples and then converts input data. The ADC_CLK signal frequency f_{ADC} must be between 0.6 and 14 MHz. These frequency restrictions are defined in the datasheet [4]. The corresponding ADC_CLK period T_{ADC} range is therefore from 0.0714 μ s to 1.667 μ s.

There are three possible inputs to the conversion clock: a dedicated 14 MHz clock, the bus clock divided by two or the bus clock divided by four. The input is selected with the CKMODE field of the ADC_CFGR2 register. Table 6.3 shows the different options available for the ADC clock.

Table 6.3 Codes for ADC Clock Selection

CKMODE	Clock Selection
00	Dedicated clock
01	Bus clock divided by 2
10	Bus clock divided by 4
11	Reserved

Analog Input Channels

The input multiplexer can select one of 19 input channels. There are several special multiplexer inputs, listed below. Using these channels requires enabling them through the ADC Common Configuration Register (ADC_CCR), which is described in the reference manual [3].

- Channel 16 is connected to an on-chip temperature sensor.
- Channel 17 is connected to an internal reference voltage V_{REFINT} (nominally 1.23 V).
- Channel 18 is connected to a voltage divider which delivers $\frac{1}{2}$ of the voltage on the external V_{BAT} pin.

Each input channel x has a field CHSEL x in the ADC_CHSELR register (Figure 6.23); setting it to 1 indicates the channel is to be converted. If multiple CHSEL x bits are set, then the ADC will convert each of them as it scans through the channel list to perform a sequence of conversions. This is explained further below in the section **Advanced ADC Features**.

To be used as ADC inputs, the GPIOs need to be configured in analog mode by setting the appropriate MODER x bits to 11. Pin-out information for analog inputs is summarized in Table 6.4. Remember that the data sheet provides information on connections between ADC channels and MCU pins, and the Nucleo-F091RC manual explains how MCU pins are connected to the board's header connectors.

Conversion Trigger

A trigger event starts the conversion process. The trigger can be a hardware signal or software write, determined by the EXTEN bits in ADC_CFGR1 (shown in Figure 6.24). A trigger event may cause one or multiple conversions, as explained further below in the section **Advanced ADC Features**.

- Software triggering is selected when EXTEN is 00. To trigger a conversion, write a 1 to ADSTART. This is typically done by software but could also be performed by the direct memory access peripheral (discussed further in Chapter 9).
- Hardware triggering is selected when EXTEN is not 00. Software must write a 1 to ADSTART to enable the trigger. The value of EXTEN then determines which signal edges trigger a conversion: rising edge (01), falling edge (10) or both edges (11). The EXTSEL field of the register ADC_CFGR1 selects the hardware trigger signal, all of which come from timers as shown in Table 6.5.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CHSEL 18	CHSEL 17	CHSEL 16
Reset														0	0	0
Access														rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	CHSEL 15	CHSEL 14	CHSEL 13	CHSEL 12	CHSEL 11	CHSEL 10	CHSEL 9	CHSEL 8	CHSEL 7	CHSEL 6	CHSEL 5	CHSEL 4	CHSEL 3	CHSEL 2	CHSEL 1	CHSEL 0
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 6.23 Set a channel's field to 1 in the ADC_CHSELR register to indicate it should be converted.

Table 6.4 ADC Inputs and Pin Assignment

ADC channel	MCU signal	Nucleo-64 Header and Pin
0	PA0	CN8 pin 1, CN7 pin 28
1	PA1	CN8 pin 2, CN7 pin 30
2	PA2	CN10 pin 35
3	PA3	CN10 pin 37
4	PA4	CN8 pin 3, CN7 pin 32
5	PA5	CN10 pin 11
6	PA6	CN10 pin 13
7	PA7	CN10 pin 15
8	PB0	CN8 pin 4, CN7 pin 34
9	PB1	CN10 pin 24
10	PC0	CN8 pin 6, CN7 pin 38
11	PC1	CN8 pin 5, CN7 pin 36
12	PC2	CN7 pin 35
13	PC3	CN7 pin 37
14	PC4	CN10 pin 34
15	PC5	CN10 pin 6

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	AWDCH					Res.	Res.	AWD EN	AWD SGL	Res.	Res.	Res.	Res.	Res.	DISC EN
Reset		0	0	0	0	0			0	0						0
Access		rw	rw	rw	rw	rw			rw	rw						rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	AUT OFF	WAIT	CONT	OVR MOD	EXTEN		Res.	EXTSEL			ALIGN	RES		SCAN DIR	DMA CFG	DMA EN
Reset	0	0	0	0	0	0		0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 6.24 ADC_CFGR1 controls key characteristics of the ADC.

SAR Converter

The ADC in the STM32F091RC MCU first samples and holds the selected input channel and then uses successive approximation to quantize its voltage. The time required to perform an ADC conversion is the sum of the sampling time T_{SMPL} and the SAR operation time T_{SAR} .

The SMP field controls the sampling time in ADC clock cycles, from 1.5 (SMP=000) to 239.5 (SMP=111) ADC clock cycles. Assuming $f_{\text{ADC}} = 14 \text{ MHz}$, T_{SMPL} can be set from $0.107 \mu\text{s}$ to $17.107 \mu\text{s}$.

The RES field of ADC_CFGR1 determines conversion resolution. Table 6.6 shows how the RES field determines resolution and T_{SAR} conversion time.

Table 6.5 Codes for Selecting Hardware Trigger Signal for the ADC

EXTSEL	Trigger selected for ADC
000	TIM1_TRGO
001	TIM1_CC4
010	TIM2_TRGO
011	TIM3_TRGO
100	TIM15_TRGO
101	Reserved
110	Reserved
111	Reserved

Note that ADSTART must be set to 1 to allow conversions to begin, whether immediately (with software triggering) or with the first valid trigger signal edge (with hardware triggering). Conversion can be stopped by writing 1 to ADSTP in ADC_CR.

Table 6.6 ADC resolution and SAR conversion time, assuming $f_{\text{ADC}} = 14 \text{ MHz}$ and $T_{\text{SMPL}} = 1.5T_{\text{ADC}}$.

RES	Resolution	TSAR Conversion Clocks Needed	TConv = TSMPL + TSAR
00	12 bits	$12.5 T_{\text{ADC}}$	$1.000 \mu\text{s}$
01	10 bits	$11.5 T_{\text{ADC}}$	$0.928 \mu\text{s}$
10	8 bits	$9.5 T_{\text{ADC}}$	$0.785 \mu\text{s}$
11	6 bits	$7.5 T_{\text{ADC}}$	$0.643 \mu\text{s}$

The ADC_DR data register holds the result of the most recent conversion. Figure 6.25 shows how the data is laid out in ADC_DR based upon the RES and ALIGN fields of ADC_CFGR1. Note that the data can be left- or right-aligned.

Low-power conversions can be achieved by reducing the maximum ADC clock speed, by selecting the Wait mode or the Auto-off mode of the ADC. You can change the clock mode by

ALIGN	RES	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0x0	0	0	0	0	DR[11:0]											
	0x1	0	0	0	0	0	0	DR[9:0]									
	0x2	0	0	0	0	0	0	0	0	DR[7:0]							
	0x3	0	0	0	0	0	0	0	0	0	0	DR[5:0]					
1	0x0	DR[11:0]												0	0	0	0
	0x1	DR[9:0]										0	0	0	0	0	0
	0x2	DR[7:0]								0	0	0	0	0	0	0	0
	0x3	0	0	0	0	0	0	0	0	DR[5:0]						0	0

Figure 6.25 Format of data in ADC_DR data result register is controlled on selected resolution and alignment specific in ADC_CFGR1.

writing the CKMODE bits in ADC_CFGR2. You can select the Wait mode by setting the WAIT field of ADC_CFGR1. In this case, a new conversion can start only if the previous conversion has been handled.

The Auto-off mode powers off the ADC between conversions. It is enabled by setting the AUTOFF field in the ADC_CFGR1 register.

Status and Detecting Conversion Completion

A completed conversion is indicated by the control hardware setting the End Of Conversion (EOC) bit in the Interrupt and Status Register (ADC_ISR) to one. This will generate an interrupt if the field EOCIE in the ADC Interrupt Enable Register (ADC_IER) is one. Otherwise EOC must be polled by software, or DMA must be used. Note that the ADC_IER has fields to allow up to six types of events to generate an ADC interrupt.

The ADC IRQ handler is called `ADC1_COMP_IRQHandler` and is shared with the two analog comparators. If the ADC and a comparator are used to generate interrupts, then the handler must determine which peripheral to service.

Advanced ADC Features

Sequences of Conversions

To improve ADC throughput and system responsiveness, it can perform a **sequence** of ADC conversions on different input channels without the need for software to select the next channel.

ADC_CHSELR is configured (by setting appropriate CHSELx bits) to select the channels to convert in the sequence. The ADC samples and converts channels with their CHSELx bit set. The SCANDIR field in ADC_CFGR1 determines the order in which channels are converted. With a value of 0, the channels will be converted in order of increasing channel number, while a 1 results in a decreasing channel number order.

Interrupts can be generated after each channel conversion completes (if EOCIE is set), and also after the entire sequence completes. The End Of Sequence (EOS) bit indicates the end of a conversion sequence when 1, and will generate an interrupt if EOSIE in ADC_IER is set to 1.

Discontinuous Triggering within a Sequence of Conversions

A single trigger event can cause just one or all of the conversions in a sequence. Consider a situation with ten CHSELx bits set in ADC_CHSELR. A conversion sequence will consist of ten conversions. Do we want a trigger event for each conversion, so ten trigger events are needed to convert all channels? Or do we want a single trigger event to cause all ten conversions back-to-back? The first approach uses the ADC more efficiently, while the second precisely synchronizes each conversion with its trigger. For some applications, one of the approaches may be a much better match.

These approaches reduce software overhead at the expense of loosening the timing connection between the trigger event and a given channel's conversion. If better timing precision is needed, the discontinuous conversion mode uses a trigger event for each channel in the sequence.

Both approaches are supported by the ADC. The first mode is called Discontinuous Conversion Mode and is selected by setting the DISCEN bit in ADC_CFGR1. The second mode (non-discontinuous?) is the default and is selected by clearing DISCEN. For both modes, CHSEL and SCANDIR determine which channels are converted and in which order.

Single vs. Continuous Conversion of Sequences

After a conversion sequence has been completed, the ADC can await the next trigger, or it can begin the next conversion sequence automatically. The first is called Single Mode, and the latter is Continuous Conversion Mode and is enabled by setting the field CONT in ADC_CFGR1 to one. Conversion can be stopped by writing 1 to ADSTP in ADC_CR.

Note that it is not possible to enable both continuous and discontinuous modes.

Analog Window Watchdog

The ADC has dedicated hardware that can process the results from the SAR and greatly reduce the software processing needed in many cases.

The analog window watchdog hardware can detect conversion results that are outside of a defined range and then generate an interrupt. The analog window watchdog is enabled by setting AWDEN in ADC_CFGR1 and enabling its interrupt by setting AWDIE in ADC_IER. The ADC threshold register ADC_TR sets higher (HT) and lower thresholds (LT). Only conversions with a result higher than HT or lower than LT can generate interrupts.

Either a single channel or all channels can be compared against the two thresholds. To enable the comparison for all the channels AWDSDL in ADC_CFGR1 need to be zeroed. If AWDSDL is one then the comparison is only done for the channel selected in the AWDCH field.

Example Applications

Next we will examine two applications of the ADC. Both use polling to determine when the conversion is complete, but in the next chapter we will use the ADC's interrupt to reduce processor overhead and simplify multitasking.

Hotplate Temperature Sensor

We can measure the temperature of the hotplate using a device called a thermistor, which is a sensor whose resistance varies with temperature. One type of thermistor (called negative temperature coefficient, or NTC) has a resistance that falls with increasing temperature. Figure 6.26 shows an example of an NTC thermistor whose resistance at 25°C is 33 kΩ. The manufacturer provides this information in the device's data sheet.

We can create a voltage divider with an NTC thermistor and a fixed resistor, as shown in Figure 6.27. The output voltage V_{Temp} will depend on the temperature as shown in Figure 6.28.

How can we convert this voltage reading to a temperature? Rather than use a look-up table, let's use a spreadsheet program to create an equation through a process called curve-fitting. Given the ADC conversion result n , a 12-bit conversion and $V_{\text{Ref}} = 3.3 \text{ V}$, this equation will calculate the approximate temperature in degrees Celsius:

$$\begin{aligned} \text{Temperature} = & -36.9 + 0.249 * n - 3.67 * 10^{-4} * n^2 + 2.95 * 10^{-7} * n^3 \\ & - 1.21 * 10^{-10} * n^4 + 2.45 * 10^{-14} * n^5 - 1.91 * 10^{-18} * n^6 \end{aligned}$$

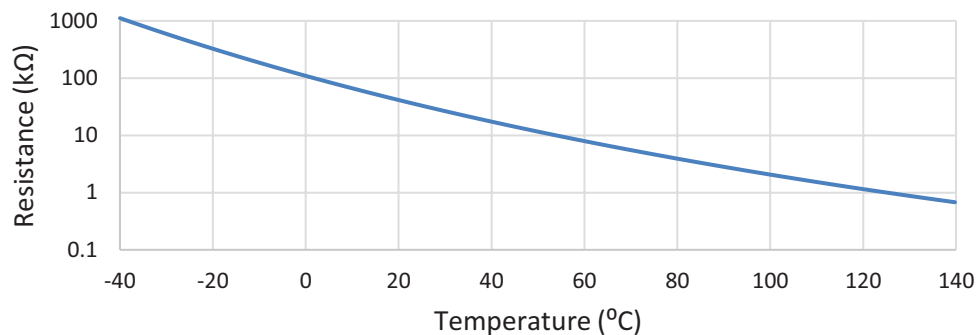


Figure 6.26 Resistance of NTC resistor falls with rising temperature. Note that the vertical axis is logarithmic.

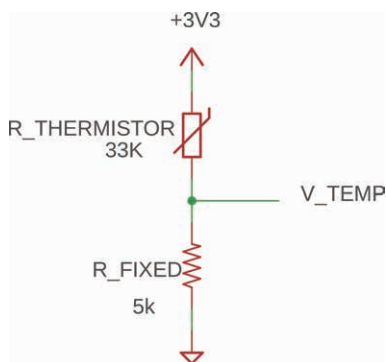


Figure 6.27 Circuit to create voltage divider with NTC thermistor and fixed resistor.

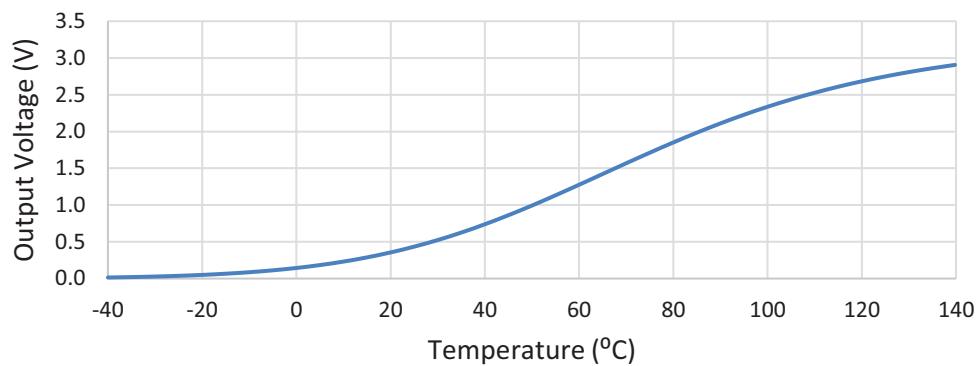


Figure 6.28 Output voltage of 3.3 V divider created with 33 kΩ NTC resistor (upper leg) and a 5 kΩ fixed resistor (lower leg).

We will connect V_{Temp} to pin PA0, which will send the signal through PA0 to ADC channel 0. The code to initialize the ADC appears in Listing 6.6.

```
void Init_ADC(void) {
    // Enable peripheral clock of ADC
    RCC->APB2ENR |= RCC_APB2ENR_ADCEN;
    // Enable peripheral clock of GPIOA
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;
    // Configure PA0 as analog input to ADC
    // (ESF_GPIO_MODER_ANALOG = 3)
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER0, ESF_GPIO_MODER_ANALOG);

    // Oscillator: Enable and Select HSI14 (from STM32F0 Reference Manual, A.7.4)
    RCC->CR2 |= RCC_CR2_HSI14ON; /* (2) Start HSI14 RC oscillator */
    while ((RCC->CR2 & RCC_CR2_HSI14RDY) == 0) { /* (3) Wait HSI14 is ready */
        /* For robust implementation, add here time-out management */
    }
    /* Select HSI14 with CKMODE=00 */
    MODIFY_FIELD(ADC1->CFGR2, ADC_CFGR2_CKMODE, 0);
    // Init ADC1
    MODIFY_FIELD(ADC1->SMPR, ADC_SMPR_SMP, 0); // SMP = 000 for minimum sample time

    /* CFGR1: The default configuration (CFGR1 = 0) matches what we want:
       some features are disabled (analog watchdog, discontinuous conversion
       mode, auto-off mode, wait conversion mode, continuous conversion mode,
       hardware trigger) and other features are selected: software trigger,
       right-aligned data, 12-bit resolution. */
    ADC1->CFGR1 = 0;

    ADC1->CHSELR = ADC_CHSELR_CHSEL0; // Select ADC input channel 0

    // Enable ADC (from STM32F0 Reference Manual, A.7.2)
    if ((ADC1->ISR & ADC_ISR_ADRDY) != 0) { /* (1) Ensure that ADRDY = 0 */
        ADC1->ISR |= ADC_ISR_ADRDY; /* (2) Clear ADRDY */
    }
    ADC1->CR |= ADC_CR_ADEN; /* (3) Enable the ADC */
    while ((ADC1->ISR & ADC_ISR_ADRDY) == 0) { /* (4) Wait until ADC ready */
        /* For robust implementation, add here time-out management */
    }
}
```

Listing 6.6 Code to initialize ADC for 12-bit conversion of channel 0.

The code to read the ADC and calculate the temperature appears in Listing 6.7. The code starts a conversion (on channel 0, which was specified in `Init_ADC`) and uses polling to determine when the conversion is complete. It then reads the ADC result and calculates the temperature using the polynomial approximation of the equation above.

```
float k0 = -36.9, k1 = 0.249, k2 = -3.67E-4, k3 = 2.95E-7,
      k4 = -1.21E-10, k5 = 2.45E-14, k6=-1.91E-18;
```

```
float Measure_Temperature(void) {
    float temp;
    int n;
```

```

// Start the conversion
ADC1->CR |= ADC_CR_ADSTART;
// Busy wait until the conversion is done
while (!(ADC1->ISR & ADC_ISR_EOC));
// Read the ADC value
n = ADC1->DR;
// Convert the voltage to Celsius
temp = k0 + n * (k1 + n * (k2 + n * (k3 + n * (k4 + n * (k5 + n * k6))));
return temp;
}

```

Listing 6.7 Code to read ADC and convert result to Celsius temperature value.

Infrared Proximity Sensor

We can use the ADC to create a sensor that uses reflected *infrared* (IR) light to detect if an object is nearby. The sensor uses an IR emitter (LED) and an IR detector (phototransistor) pointing in the same direction, as shown in Figure 6.29. If there is no object in front of the sensor, then no IR energy will be reflected back to the detector. If an object is present, then there will be a reflection and the detector will see it. The strength of the reflection depends on the object's distance, size, reflectivity, and orientation.

infrared (IR)

Electromagnetic energy immediately past the visible portion of the spectrum; also called invisible light

The proximity sensor works with a combination of hardware and software. Simply keeping the emitter on and measuring the detector's signal will not work well because the system will be very vulnerable to changes in ambient light levels. We will use a more sophisticated approach that compares the IR levels with the emitter off and on in order to subtract out the effects of ambient light.

Sensing occurs in two steps: First, the software measures the IR light level (using IR-sensitive phototransistor Q1 and the ADC) when the IR-emitting LED is turned **off**. Second, the software measures the IR light level when the IR LED is turned **on**. An object that reflects the IR back will increase this IR brightness level. The difference between the two readings indicates the reflected signal's strength.

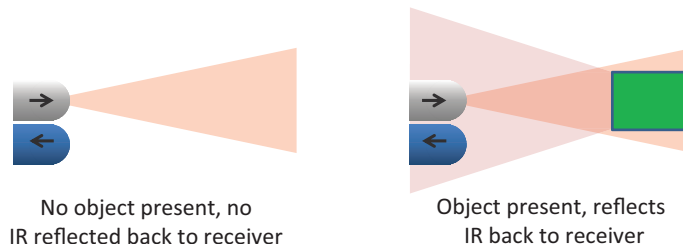


Figure 6.29 Proximity sensor method of operation.

Circuit Description

The circuit is shown in Figure 6.30 and Figure 6.31. The IR energy is emitted by IR LED D1, which the MCU controls with a GPIO pin output called IR_LED_DRIVE. The IR energy is detected by an IR-sensitive phototransistor Q1. Q1 forms a voltage divider with R2. A higher level of IR energy lowers the phototransistor's resistance and therefore lowers the voltage on signal IR_SENSE.

The traces in Figure 6.32 show the operation of the circuit. The IR LED is on when the upper trace is low and off when it is high. There is no reflecting object present, but the IR LED emits a small amount of energy laterally. This IR energy strikes the phototransistor, resulting in a minor signal.

Figure 6.33 shows the circuit's behaviour with an object about 5 cm away, while Figure 6.34 shows the results from an object about 1 cm away. Note that the phototransistor takes time to respond to the change in IR energy, as shown by the curves in the lower traces. Our software must

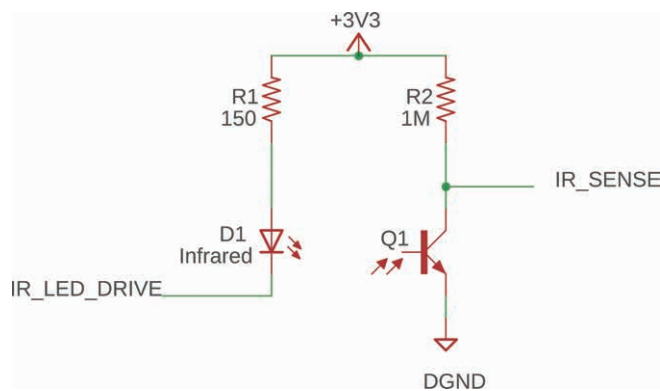


Figure 6.30 Schematic diagram of infrared proximity sensor circuit.

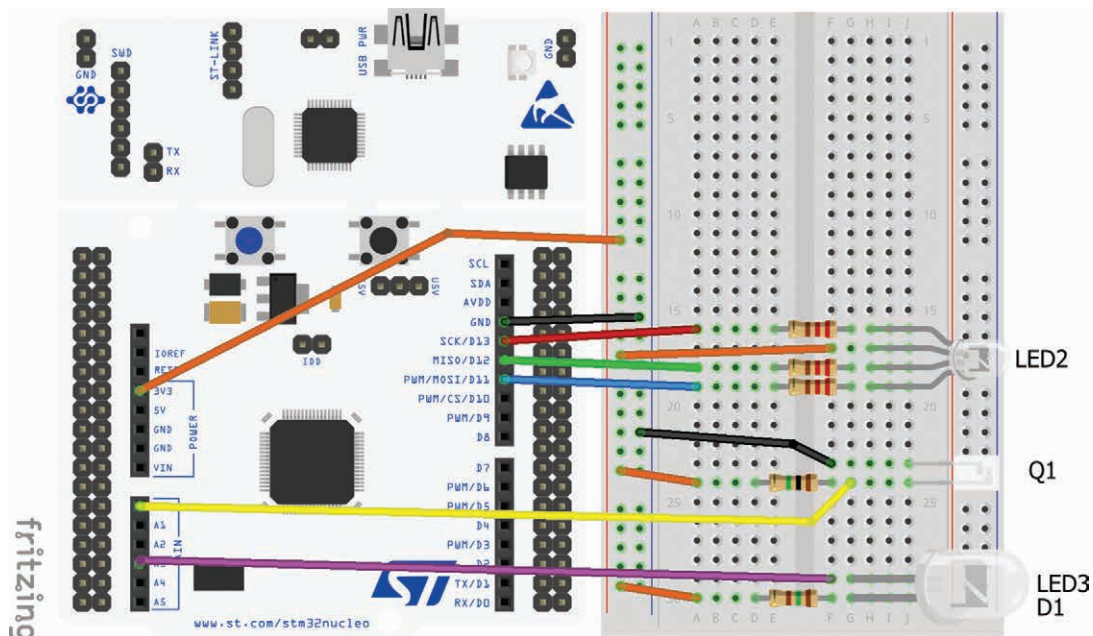


Figure 6.31 Parts placement of infrared proximity sensor circuit with RGB LED output.

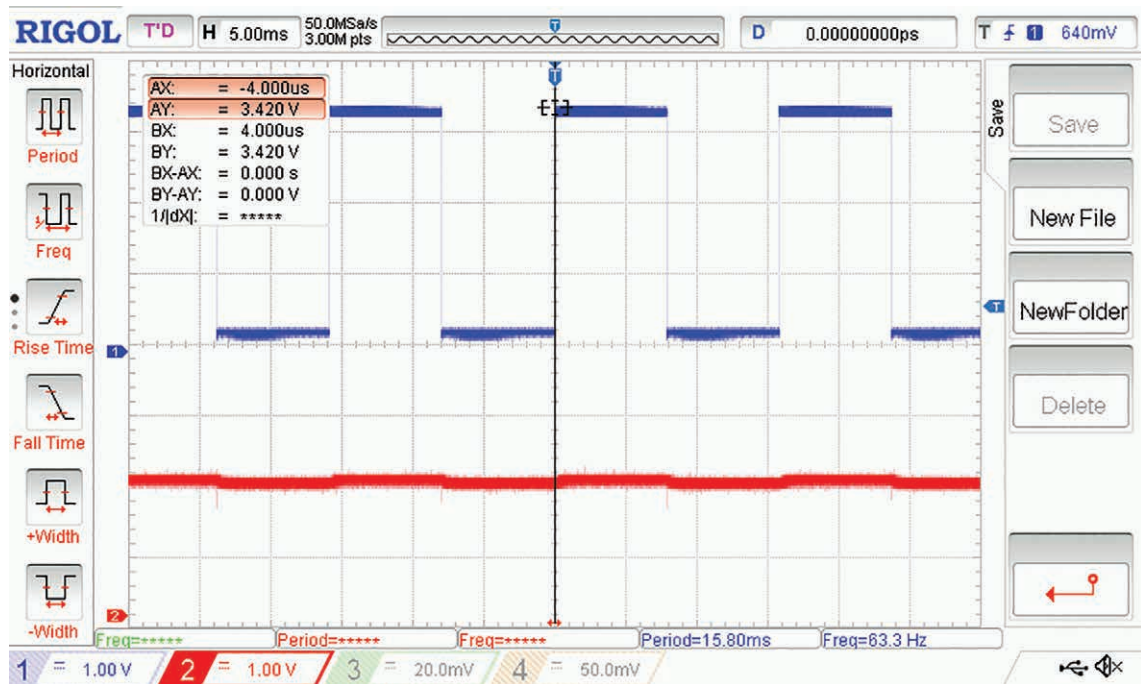


Figure 6.32 No reflecting object nearby. Upper trace is LED (transmitter) drive signal, lower trace is receiver (phototransistor) signal.

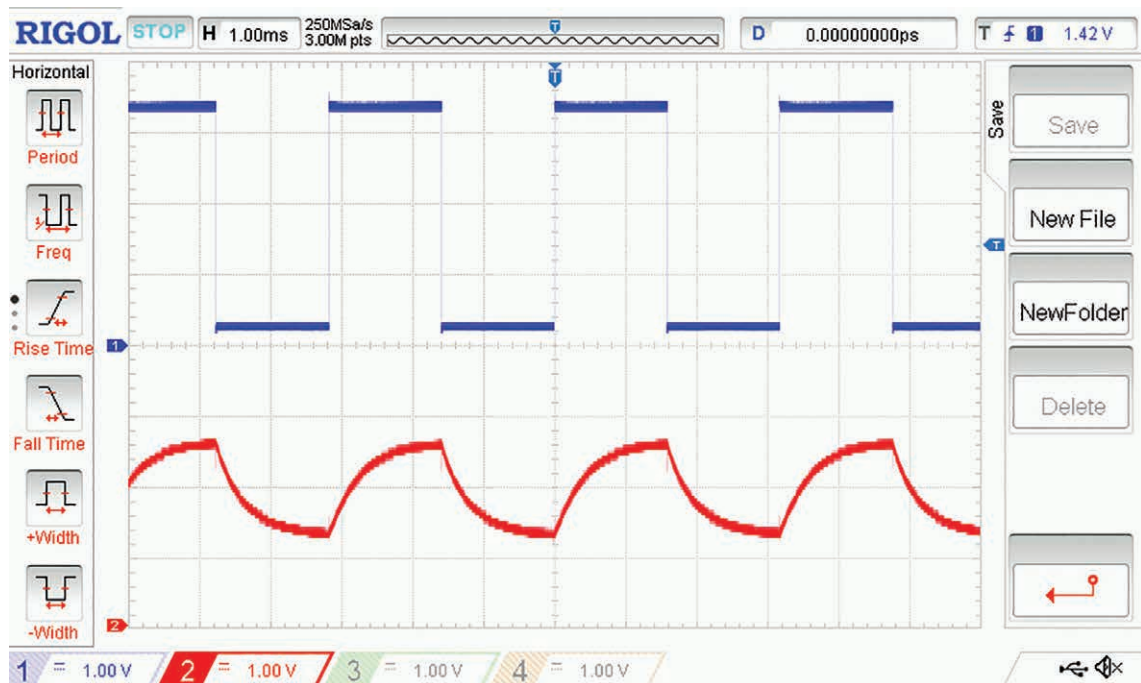


Figure 6.33 Transmitter and receiver signals with reflecting object 5 cm away.

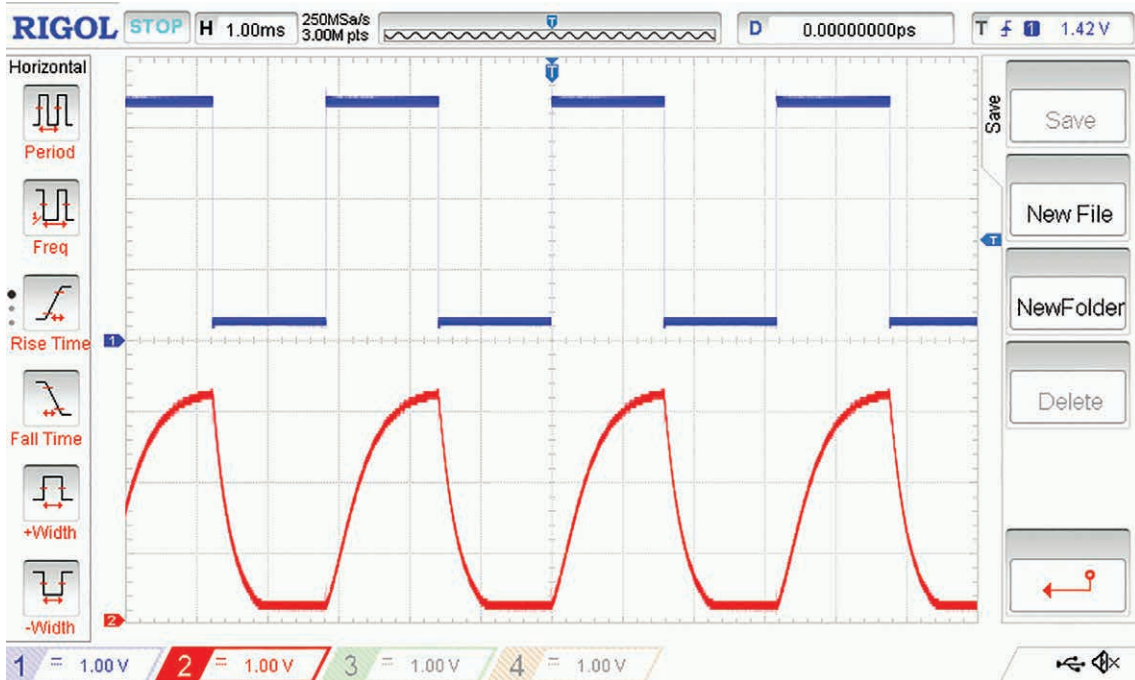


Figure 6.34 Transmitter and receiver signals with reflecting object 1 cm away.

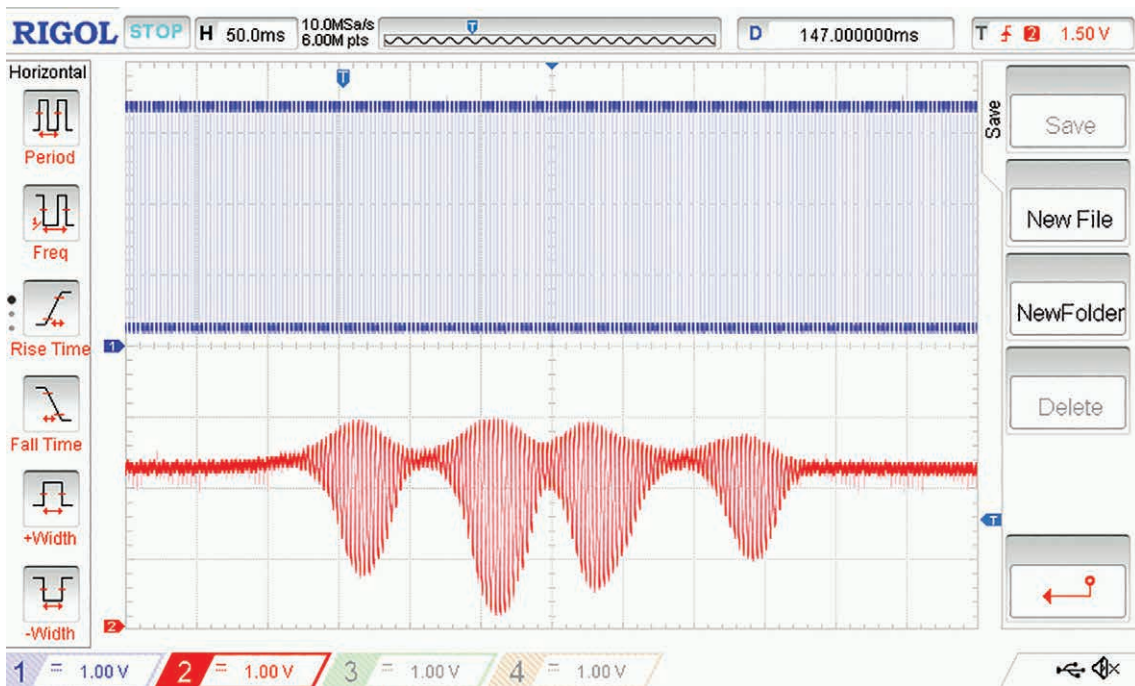


Figure 6.35 Transmitter and receiver signals detecting four fingers passing over proximity sensor in sequence.

wait after switching IR_LED_DRIVE before it measures IR_SENSE. The longer it waits, the more sensitive the system will be.

Figure 6.35 shows the result of sweeping four separated fingers over the proximity sensors. The oscilloscope has been adjusted to show a longer time period.

Control Software

The control software uses several functions, data structures and symbol definitions and to do its work.

```
#define IR_LED_OFF_MSK (GPIO_BSRR_BS_0)
#define IR_LED_ON_MSK (GPIO_BSRR_BR_0)

void Init_IR_LED(void) {
    // Enable peripheral clock of GPIOB
    RCC->AHBENR |= RCC_AHBENR_GPIOBEN;
    // Init PB0 as digital output
    MODIFY_FIELD(GPIOB->MODER, GPIO_MODER_MODER0, ESF_GPIO_MODER_OUTPUT);
    // Turn off IR LED
    GPIOB->BSRR = IR_LED_OFF_MSK;
}
```

Listing 6.8 Code to initialize the IR LED.

We will reuse the previous `Init_ADC` function (in Listing 6.6) to configure the GPIO pin PA0 and ADC input. The `Init_IR_LED` function (in Listing 6.8) configures the GPIO pin PB0 to drive the IR LED.

```
void Control_IR_LED(unsigned int led_on) {
    GPIOB->BSRR =
        led_on ? IR_LED_ON_MSK : IR_LED_OFF_MSK;
}

unsigned int Measure_IR(void) {
    unsigned int n;

    // Start the conversion
    ADC1->CR |= ADC_CR_ADSTART;
    // Busy wait until the conversion is done
    while (!(ADC1->ISR & ADC_ISR_EOC))
        ;
    // Read the ADC value
    n = ADC1->DR;
    // We want return number to rise with increasing
    // IR but voltage falls with increasing IR.
    // So subtract result from max possible value
    // (0xffff for 12-bit conversion)
    return 0xffff - n;
}
```

Listing 6.9 Code to control IR LED and measure IR level from phototransistor through ADC.

The `Control_IR_LED` function (in Listing 6.9) turns on or off the IR LED based on the function argument. The `Measure_IR` function starts an ADC conversion, blocks (waits) until the conversion is complete, reads the result, and then inverts the result so that larger values indicate brighter levels.

```
void ShortDelay(volatile unsigned int time_del) {
    volatile int n;
    while (time_del--);
}
```

Listing 6.10 Simple busy-waiting time delay function.

The `ShortDelay` function (in Listing 6.10) waits for an amount of time proportional to the input parameter. Note that the actual time delay is not specified, and will vary based on processor speed, compiler settings, and other factors.

```
#define RED (0)
#define GREEN (1)
#define BLUE (2)
#define NUM_RANGE_STEPS (7)

const int Threshold[NUM_RANGE_STEPS] =
    { 500, 200, 80, 30, 15, 10, 0 };

const int Colors[NUM_RANGE_STEPS][3] = {
    {1, 1, 1},    // white
    {1, 0, 1},    // magenta
    {1, 0, 0},    // red
    {1, 1, 0},    // yellow
    {0, 0, 1},    // blue
    {0, 1, 0},    // green
    {0, 0, 0}     // off
};

void Display_IR_Reflectance(int b) {
    unsigned int i;

    for (i = 0; i < NUM_RANGE_STEPS - 1; i++) {
        if (b > Threshold[i])
            break; // break out of for loop
    }
    Control_RGB_LEDs(Colors[i][RED],
                    Colors[i][GREEN],
                    Colors[i][BLUE]);
}
```

Listing 6.11 Code to light RGB LED based on IR reflectance.

The `Display_IR_Reflectance` function (in Listing 6.11) lights the RGB LED according to the input argument (IR brightness difference), and uses a table to define thresholds and colors.

```
#define T_DELAY_ON (400)
#define T_DELAY_OFF (400)
#define NUM_SAMPLES_TO_AVG (20)

int main(void) {
    /* Making local variables static makes
       them visible to debugger */
    static int on_brightness=0, off_brightness=0;
    static int avg_diff, diff;
    unsigned int n;

    Init_ADC();
    Init_GPIO_RGB();
    Init_IR_LED();
    Control_RGB_LEDS(0, 0, 0);

    while (1) {
        diff = 0;
        for (n = 0; n < NUM_SAMPLES_TO_AVG; n++) {
            // Measure IR level with IRLED off
            Control_IR_LED(0);
            ShortDelay(T_DELAY_OFF);
            off_brightness = Measure_IR();
            // Measure IR level with IRLED on
            Control_IR_LED(1);
            ShortDelay(T_DELAY_ON);
            on_brightness = Measure_IR();
            // Compute difference
            diff += on_brightness - off_brightness;
        }
        // Compute average difference
        avg_diff = diff / NUM_SAMPLES_TO_AVG;
        // Light RGB LED according to reflectance
        Display_IR_Reflectance(avg_diff);
    }
}
```

Listing 6.12 Main function that controls IR LED, measures IR phototransistor voltage, and calculates and displays reflectance.

The `main` function (in Listing 6.12) initializes the system and then measures IR reflectance. To do this, it repeatedly measures the difference in brightness caused by lighting the IR LED and then calls a function to indicate the reflectance with color-coding on the RGB LED. In order to reduce noise, it averages multiple measurements (`NUM_SAMPLES_TO_AVG`) before each update of the LED color.

Summary

In this chapter we have seen how a digital microcontroller can measure and generate analog signals. We began by examining quantization and sampling. We then examined various peripherals. A digital-to-analog converter allows the MCU to generate an analog signal. A comparator allows the MCU to determine which of the two analog voltages is greater. Using a known reference voltage as one of the inputs allows us to determine whether the other input is above or below that voltage. An analog-to-digital converter measures an analog voltage and provides a proportional digital representation.

Exercises

For all of these questions, assume the STM32F091RC peripherals are used unless specified otherwise.

1. Consider a 12-bit ADC with a reference voltage of 3.3 V operating in single-ended mode. Given an input voltage of 0.92 V, what will the output code be?
2. Consider an 8-bit ADC with a positive reference voltage of 2.7 V operating in single-ended mode. What input voltage range will lead to an output code of 0x34?
3. Consider a 12-bit ADC with an unknown positive reference voltage operating in single-ended mode. What is the positive reference voltage if sampling the 1.23 V internal voltage reference results in a code of 0x513?
4. Consider a 12-bit ADC with a reference voltage of 3.3 V operating in single-ended mode. If it samples the internal temperature sensor and reads a voltage of 0.621 V, what is the temperature? Assume $V_{30} = 1.43$ V and $\text{Avg_Slope} = 4.3$ mV/°C. Refer to Section 6.3.19 (Temperature sensor characteristics) in the device data sheet as needed [5].
5. Consider a 12-bit DAC with a reference voltage of 3.3 V. What input code will result in an output of 1.43 V?
6. Consider an 8-bit DAC with a reference voltage of 2.7 V. Given that the input code is 0x104, what is the output voltage?
7. What is the output voltage resolution of an 8-bit DAC with a reference voltage of 3.0 V?
8. How would you configure the comparator in the STM32F091RC to trigger whenever the input voltage rises above 2.0 V? Assume the reference voltage is 3.3 V.

References

- [1] L. Tan and J. Jian, *Digital Signal Processing: Fundamentals and Applications*, 2nd ed., Elsevier Inc., 2013.
- [2] C. L. Philips, T. Nagle and A. Chakraborty, *Digital Control System Analysis and Design*, 4th ed., Pearson, 2014.
- [3] STMicroelectronics NV, *Reference Manual RM0091: STM32F0x1/STM32F0x2/STM32F0x8*, 2017.
- [4] STMicroelectronics NV, *User Manual UM1724: STM32 Nucleo-64 Boards*, 2019.
- [5] STMicroelectronics NV, *STM32F091xB STM32F091xC Data Sheet*, DocID 026284, 2017.